

Module 2: Syntax Analysis

Question 3 (3 marks each):

- A-3: With an example, write the steps to remove left recursion.
- B-3: Given the grammar $S \rightarrow (L) \mid a, L \rightarrow L, S \mid S$:
 - (i) Is the grammar ambiguous? Justify.
 - (ii) Build a parse tree for the string $(a, ((a, a), (a, a)))$.
- D-3: What is left recursion? Explain the rule to eliminate left recursion.
- E-3: Write the algorithm to remove left recursion and remove left recursion from the grammar:
 $E \rightarrow E \oplus E \mid 00, S \rightarrow S11 \mid 1$.
- F-3: Differentiate between syntax tree and parse tree with an example.
- July 2023-3: With an example, write the steps to remove left recursion.

Question 4 (3 marks each):

- A-4: Find the FIRST and FOLLOW sets of each nonterminal in the grammar:
 $E \rightarrow EAE \mid (E) \mid -E \mid id, A \rightarrow + \mid \times$.
- B-4: Compute the FIRST and FOLLOW sets for the grammar:
 $S \rightarrow SS \mid AB, A \rightarrow Aa \mid a, B \rightarrow Bb \mid b$.
- D-4: Describe the sets-of-items construction algorithm for an SLR parser.
- E-4: Write the algorithm for left factoring. Left factor the grammar:
 $S \rightarrow abAA \mid ab, A \rightarrow abA \mid ab$.
- F-4: Show that the grammar $S \rightarrow \emptyset S1 \mid SS \mid \epsilon$ is ambiguous or not.
- July 2023-4: What is left factoring? Left factor the grammar:
 $E \rightarrow E + T \mid T, T \rightarrow float \mid float * T \mid (E)$.

Question 13 (14 marks each):

- A-13:
 - (i) Show that the grammar $S \rightarrow iCtSeS \mid iCtS \mid b, C \rightarrow a$ is ambiguous. (ii) Eliminate ambiguity from the grammar.
 - b) Construct a recursive descent parser for handling arithmetic expressions.
- B-13:
 - a) For the grammar $E \rightarrow E + T \mid T, T \rightarrow T * F \mid F, F \rightarrow \sim F \mid (E) \mid id$:
 - (i) Remove left recursion.
 - (ii) Construct a predictive parsing table.
 - (iii) Justify the statement "The grammar is LL(1)".
 - b) Design a recursive descent parser for the grammar: $S \rightarrow cAd, A \rightarrow ab \mid b$.
- D-13:
 - a) Illustrate the design of a recursive descent parser with a suitable grammar.
 - b) Explain any two drawbacks of a recursive descent parser and their solutions.
- E-13:
 - a) For the grammar $S \rightarrow (L) \mid a, L \rightarrow L, S \mid S$:
 - (i) Remove left recursion.
 - (ii) Construct a predictive parsing table.
 - (iii) Justify the statement "The grammar is LL(1)".
 - b) Write the recursive descent parsing procedure for the grammar:
 $S \rightarrow iCS' \mid a, S' \rightarrow eS \mid \epsilon, C \rightarrow b$.
- F-13:
 - a) Write the algorithm and eliminate left recursion from the grammar:
 $S \rightarrow a \mid \wedge \mid (T), T \rightarrow T, S \mid S$.
 - b) Explain recursive descent parser with an example.
- July 2023-13:
 - a) Find the FIRST and FOLLOW sets of each nonterminal in the grammar:
 $S \rightarrow aB0h \mid bBc, B \rightarrow eC, C \rightarrow bC \mid \epsilon, D \rightarrow EF, E \rightarrow g \mid \epsilon, F \rightarrow f \mid \epsilon$.
 - b) Explain the error recovery strategies in parsing.

Question 14 (14 marks each):

- A-14:
 - a) Write the non-recursive predictive parsing algorithm.
 - b) Prove that the grammar $S \rightarrow iEtSS' \mid a, S' \rightarrow eS \mid \epsilon, E \rightarrow b$ is not LL(1).
- B-14:
 - a) Left factor the grammar $S \rightarrow TL; T \rightarrow int \mid float, L \rightarrow L, id \mid id$, and obtain the LL(1) parsing table. Is the grammar LL(1)? Justify.
 - b) Write all the moves by the LL(1) parser for parsing the input $int\ id, id;$.
- D-14:
 - a) Write and explain the algorithm for constructing an LL(1) predictive parsing table.
 - b) Construct a predictive parsing table for the grammar:
 $S \rightarrow SAaAb \mid SBbBa \mid \epsilon, A \rightarrow t, B \rightarrow c$.
- E-14:
 - a) Write algorithms for finding FIRST and FOLLOW sets. What is the role of FIRST and FOLLOW in a predictive parser?
 - b) Compute the FIRST and FOLLOW sets for the nonterminals in the grammar:
 $S \rightarrow SDB \mid DbB \mid Ba, A \rightarrow da \mid BD, B \rightarrow g \mid \epsilon, D \rightarrow h \mid \epsilon$.
- F-14:
 - a) Find the FIRST and FOLLOW sets of the grammar:
 $S \rightarrow ACB \mid CbB \mid Ba, A \rightarrow da \mid BC, B \rightarrow g \mid \epsilon, C \rightarrow h \mid \epsilon$.
 - b) Construct a predictive parsing table for the grammar:
 $S \rightarrow (L) \mid a, L \rightarrow SL' \mid L', L' \rightarrow SL' \mid \epsilon$. Check whether the grammar is LL(1) or not. Justify.
- July 2023-14:
 - a) (i) Show that the grammar $E \rightarrow E + E \mid E - E, E \rightarrow E * E \mid E / E, E \rightarrow E^A E, E \rightarrow (E) \mid id$ is ambiguous or not. (ii) Eliminate ambiguity from the grammar (precedence: id, (), ^, *, /, +, -).
 - b) Construct a non-recursive predictive parsing table for the grammar:
 $S \rightarrow (L) \mid a, L \rightarrow L, S \mid S$. Prove that the grammar is LL(1) or not.

How can Grok help?

 DeepSearch  Think

Grok 3

